

```

# numpy, matplotlib, astropy dependency
from axionbloch.dependency import *
# Gyromagnetic ratio and magnetic dipole moment of Xe-129
from axionbloch.constants import gamma_Xe129, mu_Xe129
# classes for simulations
from axionbloch.SimuTools import MagField, Simulations
from axionbloch.SimuTypes import SimuParams
from axionbloch.Sample import Sample
from axionbloch.Apparatus import Magnet
from axionbloch.MilkyWayAxionHalo import MilkyWayAxionHalo
# Define the Xe-129 sample with gyromagnetic ratio,
# mass density, molar mass, number of spins per molecule,
# relaxation times, volume, magnetic dipole moment,
# temperature, and polarization.
sample = Sample(
    name="Liquid Xe-129",
    gamma=gamma_Xe129,
    massDensity=3.1 * unit.g * unit.cm ** (-3),
    molarMass=131.29 * unit.g / unit.mol,
    numOfSpinsPerMolecule=1 * unit.one,
    T2=10 * unit.minute,
    T1=15 * unit.minute,
    vol=1 * unit.cm**3,
    mu=mu_Xe129, # magnetic dipole moment
    temp=163 * unit.K,
    pol=1 * unit.percent,
    verbose=False,
)
# Define the axion field with axion Compton frequency
# and axion-nucleon coupling strength
axion = MilkyWayAxionHalo(
    name="Milky Way Axion Halo",
    nu_a=1 * unit.kHz,
    g_aNN=1.0e-9 * unit.GeV ** (-1),
    verbose=False,
)
# Set the bias field strength, direction, and homogeneity
magnet = Magnet(
    B0=axion.nu_a_eff / (sample.gamma / (2 * PI)),
    FWHM=2 * ppm,
)
# rms amplitude of pseudomagnetic field
B_a_rms = (axion.getRabiFreq() / (sample.gamma / (2 * PI))).real
# Bundle all inputs into one dictionary
params: SimuParams = {
    "key_info": {"nu_a": axion.nu_a},
    "axion": axion,
    "sample": sample,
    "magnet": magnet,
    "excField": MagField(),
    "B_a_rms": B_a_rms,
    # Number of random field realizations.
    "numFields": 1000,
    "rand_seed": 10, # random seed
    # amplitude, polar and azimuthal angle
    # of the initial magnetization
    "init_M": None, # use None when setting M automatically
    "init_M_theta": 0 * unit.rad,
    "init_M_phi": 0 * unit.rad,
    # sampling rate and duration of the time series
    "rate": 1 * unit.Hz,
    "duration": 4000 * unit.s,
}
# Create and execute the simulation job collection
simulations = Simulations(all_params=[params])
# run the simulation
simulations.run(verbose=True)
# plot results
for i, item in enumerate(simulations.pool):
    item.simu.displayTrjries()
# Save to .pkl file for later analysis
simulations.saveToPkl(dir="path_to_save")

```